Practice Problems: Object Oriented Programming 2 (OOP-2)

Code writing:

1. Write a program to print “Hello, World!”
2. Declare variables of type int, double, char, bool, and print them.
3. Take an integer from user input and print it.
4. Take two numbers and print their sum.
5. Convert temperature from Celsius to Fahrenheit.
6. Swap two numbers using a third variable.
7. Swap two numbers without using a third variable.
8. Check the data type size of int, double, and float.
9. Demonstrate implicit type casting.
10. Demonstrate explicit type casting.
11. Calculate area of a circle.
12. Calculate simple interest.
13. Check whether a number is even or odd.
14. Find the largest of two numbers.
15. Find the largest of three numbers.
16. Calculate power of a number using Math.Pow().
17. Find square root using Math.Sqrt().
18. Generate a random number between 1 and 100.
19. Check if a number is divisible by 5 and 11.
20. Compute quotient and remainder.
21. Read a string and print its length.
22. Convert string to uppercase and lowercase.
23. Check if a string contains a specific word.
24. Reverse a string.
25. Compare two strings for equality.
26. Check if a string is palindrome.
27. Count vowels in a string.
28. Replace all spaces with hyphens.

29. Extract first 5 characters of a string.
30. Concatenate first and last name.
31. Check if a number is positive, negative, or zero.
32. Check voting eligibility.
33. Check leap year.
34. Grade calculator (A, B, C, etc.).
35. Menu-driven calculator using switch.
36. Display day name from number using switch.
37. Check if a character is vowel or consonant.
38. Check if a year is century year.
39. Find absolute value.
40. Determine ticket price based on age group.
41. Print numbers from 1 to 10 using for.
42. Print numbers from 10 to 1 using while.
43. Print multiplication table of a number.
44. Sum of first N natural numbers.
45. Factorial of a number.
46. Count digits in a number.
47. Reverse digits of a number.
48. Check if number is Armstrong.
49. Print Fibonacci series up to N terms.
50. Demonstrate break and continue.
51. Create array and print elements.
52. Find sum of array elements.
53. Find average of array.
54. Find maximum element.
55. Find minimum element.
56. Reverse an array.
57. Sort array ascending.
58. Count even and odd numbers in array.
59. Search element in array (linear search).

60. Merge two arrays.
61. Create method to print greeting.
62. Method that returns sum of two numbers.
63. Method to check prime number.
64. Method to calculate factorial.
65. Method to reverse string.
66. Method returning largest of three numbers.
67. Method with no return type (void).
68. Method that returns boolean.
69. Method that takes array as parameter.
70. Call one method from another method.
71. Method using ref parameter to swap values.
72. Method using out parameter to return multiple results.
73. Demonstrate difference between ref and out.
74. Create overloaded methods to add integers and doubles.
75. Overload method to calculate area of circle and rectangle.
76. Overload method with different parameter counts.
77. Method with optional parameters.
78. Method with named arguments.
79. Recursive method for factorial.
80. Recursive Fibonacci.
81. Create a class Student with fields and display info.
82. Create object and assign values using dot operator.
83. Add method inside class to print details.
84. Create class Rectangle with area method.
85. Create multiple objects of same class.
86. Demonstrate class vs object difference in code.
87. Create class with static member.
88. Count number of objects created.
89. Class representing bank account (deposit, withdraw).
90. Create class with method returning object.

91. Default constructor demonstration.
92. Parameterized constructor.
93. Constructor overloading.
94. Create property with getter and setter.
95. Read-only property.
96. Auto-implemented properties.
97. Demonstrate private, public, protected.
98. Encapsulate student marks using property validation.
99. Class with multiple constructors and chaining.
100. Full OOP mini project: Create a LibraryBook class with:
    - private fields
    - properties
    - constructor
    - method to borrow and return
    - availability status

101. Create a Student class with ID, Name, and CGPA. Create 3 student objects and display their details.

102. Design a Course class. Each course has title, credit, and instructor name. Create objects and print course summary.

103. Create a Result class that stores marks of 5 subjects and calculates total and average.

104. Design a class OnlineClass with properties: platform, duration, and number of students. Create an object and display class info.

105. Create a Teacher class with salary and experience. Add a method to calculate yearly bonus based on experience.

106. Design a BankAccount class with account number, holder name, and balance. Implement deposit and withdraw methods.

107. Create a Loan class that calculates monthly EMI based on principal, rate, and time.

108. Create a Transaction class that stores transaction type, amount, and date. Display transaction history.

109. Create a SavingsAccount class with interest rate. Add method to calculate yearly interest.

110. Create a simple ATM simulation class that checks PIN and allows withdrawal.

111. Create a Product class with name, price, and stock. Display product details.

112. Design a ShoppingCart class that stores products and calculates total bill.

113. Create a Customer class with name and email. Assign purchased products to customer.

114. Create a DiscountCalculator class that applies discount based on purchase amount.

115. Create a Receipt class that prints item list, total, tax, and final amount.

116. Create a Car class with brand, model, and speed. Add method to accelerate.

117. Create a Driver class and assign a car object to the driver.

118. Design a FuelTank class that tracks fuel level and consumption.

119. Create a Bus class with seating capacity and passengers. Check if bus is full.

120. Create a vehicle rental system class that calculates rent based on hours used.

121. Create a Patient class with name, age, and disease. Display patient record.

122. Design a BMI calculator class that computes BMI from height and weight.

123. Create a Doctor class that stores specialization and visiting hours.

124. Create a Medicine class with name, expiry date, and price. Check if expired.

125. Create a mini hospital management class where a patient is assigned to a doctor.

126. Animal Sound System (Inheritance)

Use **inheritance** where:

- Animal is the **base class**
- Dog and Cat **inherit from Animal**

All classes contain method MakeSound() (override in derived classes).

Create objects:

- animal1
- dog1
- cat1

Call MakeSound() for each object.

127. Employee Management System (Inheritance)

Use **inheritance** where:

- Employee is the **base class**

- Manager and Developer **inherit from Employee**

Include properties and method ShowDetails().

Create objects:

- emp1
- manager1
- dev1

Display full information.

128. Vehicle Information System (Inheritance)

Use **inheritance** where:

- Vehicle is the **base class**
- Car and Bike **inherit from Vehicle**

Add method DisplayInfo().

Create objects:

- vehicle1
- car1
- bike1

Display details of all vehicles.

129. Student Record System (Inheritance)

Use **inheritance** where:

- Student is the **base class**
- Undergraduate and Postgraduate **inherit from Student**

Add method DisplayInfo().

Create objects:

- student1
- ugStudent1
- pgStudent1

Print student information.

130. Bank Account System (Inheritance)

Use **inheritance** where:

- BankAccount is the **base class**
- SavingsAccount and FixedDeposit **inherit from BankAccount**

Add method ShowAccount().

Create objects:

- account1
- savings1
- fd1

Display account details.

131. Shape Drawing Program (Inheritance + Overriding)

Use **inheritance** where:

- Shape is the **base class**
- Circle and Rectangle **inherit from Shape**

Shape has virtual method Draw() that derived classes override.

Create objects:

- shape1
- circle1
- rectangle1

Call Draw() for each.

132. School Staff System (Inheritance)

Use **inheritance** where:

- Staff is the **base class**
- Teacher and Administrator **inherit from Staff**

Add method DisplayStaffInfo().

Create objects:

- staff1
- teacher1
- admin1

Display staff information.

133. Electronic Device System (Inheritance)

Use **inheritance** where:

- Device is the **base class**
- Laptop and Smartphone **inherit from Device**

Add method ShowSpecs().

Create objects:

- device1
- laptop1
- phone1

Display device specifications.

134. Transport Fare Calculator (Inheritance)

Use **inheritance** where:

- Transport is the **base class**
- Bus and Train **inherit from Transport**

Add method CalculateFare().

Create objects:

- transport1
- bus1
- train1

Calculate and display fare.

135. Online Shopping Product System (Inheritance)

Use **inheritance** where:

- Product is the **base class**
- Book and Electronics **inherit from Product**

Add method DisplayProduct().

Create objects:

- product1
- book1
- electronics1

Display product details.

136. Online Shopping System

A company wants to design a console-based system to manage different types of **products in an online store**.

Requirements

There are two types of products: **ElectronicProduct** and **ClothingProduct**.

All products have:

productId

name

brand

basePrice

ElectronicProduct

Includes:

warrantyYears

powerUsage

Price Rule:

If warrantyYears > 2, add **15%** to the base price.

ClothingProduct

Includes:

size

material

Price Rule:

If material == "Silk", add **20%** to the base price.

Methods

Every product must have:

ShowProductInfo() → display all details with final price

CalculateFinalPrice() → compute final price

IsLuxuryProduct() → product is luxury if **final price > 10000**

Main Method:

```
class Program
{
  static void Main(string[] args)
  {
    Product[] products = new Product[3];

    products[0] = new ElectronicProduct("P-101", "Laptop", "Dell", 80000, 3, 120);
    products[1] = new ElectronicProduct("P-102", "Smartphone", "Samsung", 50000,
1, 30);
    products[2] = new ClothingProduct("P-103", "Evening Dress", "Zara", 7000, "M",
"Silk");

    foreach(Product p in products)
    {
      p.ShowProductInfo();
      Console.WriteLine();
    }
  }
}
```

137. Library Membership System

A library wants to manage different types of **membership plans**.

Membership Types

StudentMembership

PremiumMembership

Common Fields

memberId

memberName

joinYear

baseFee

StudentMembership

Fields:

universityName

borrowLimit

Rule:

If borrowLimit > 10, add **5%** to base fee.

PremiumMembership
Fields:
exclusiveEvents
readingRoomAccess (true/false)
Rule:
If readingRoomAccess == true, add **12%** to base fee.

Methods
Every membership must have:
ShowMembershipInfo()
CalculateFinalFee()
IsVIPMember() → fee greater than **3000**

Main Method:

```
class Program
{
  static void Main(string[] args)
  {
    Membership[] members = new Membership[3];

    members[0] = new StudentMembership("M-101","Rafi",2023,2000,"DIU",12);
    members[1] = new StudentMembership("M-102","Sara",2022,1800,"NSU",8);
    members[2] = new PremiumMembership("M-103","Imran",2021,2500,5,true);

    foreach(Membership m in members)
    {
      m.ShowMembershipInfo();
      Console.WriteLine();
    }
  }
}
```

138. Gym Membership System

A gym wants to manage different types of **fitness programs**.

Program Types

- PersonalTraining

- GroupTraining

Common Fields

- programId
- trainerName
- durationWeeks
- baseFee

PersonalTraining

Fields:

- sessionsPerWeek
- nutritionPlan (true/false)

Rule:

- If nutritionPlan == true, add **18%** to fee.

GroupTraining

Fields:

- groupSize
- equipmentProvided (true/false)

Rule:

- If groupSize > 20, add **10%** to fee.

Methods

ShowProgramInfo()

CalculateFinalFee()

IsEliteProgram() → final fee **> 8000**

Main Method:

```
class Program
{
   static void Main(string[] args)
   {
      FitnessProgram[] programs = new FitnessProgram[3];


      programs[0] = new PersonalTraining("G-101","Trainer Ali",8,6000,5,true);
      programs[1] = new GroupTraining("G-102","Trainer Hasan",6,4000,25,true);
      programs[2] = new PersonalTraining("G-103","Trainer Sam",10,7000,4,false);


      foreach(FitnessProgram p in programs)
      {
         p.ShowProgramInfo();
         Console.WriteLine();
      }
   }
}
```

139. Hotel Room Booking System

A hotel wants to manage different types of **room bookings**.

Room Types

- LuxuryRoom
- StandardRoom

Common Fields

- roomId
- guestName
- daysBooked
- basePricePerDay

LuxuryRoom

Fields:

- butlerService
- jacuzzi

Rule:

- If jacuzzi == true, add **25%**.

StandardRoom

Fields:

- breakfastIncluded
- roomView

Rule:

- If breakfastIncluded == true, add **10%**.

Methods

ShowRoomInfo()

CalculateFinalBill()

IsPremiumStay() → bill **> 20000**

Main Method:

class Program

```
{
  static void Main(string[] args)
  {
    Room[] rooms = new Room[3];


    rooms[0] = new LuxuryRoom("R-101","Nabila",3,6000,true,true);
    rooms[1] = new StandardRoom("R-102","Rahim",2,3000,true,"Sea");
    rooms[2] = new LuxuryRoom("R-103","Tanvir",4,7000,true,false);


    foreach(Room r in rooms)
    {
```

```
        r.ShowRoomInfo();

        Console.WriteLine();

    }

  }

}
```

140. Online Subscription Platform

A platform manages different types of **content subscriptions**.

Subscription Types

- MusicSubscription
- VideoSubscription

Common Fields

- subscriptionId
- userName
- basePrice
- durationMonths

MusicSubscription

Fields:

- audioQuality
- offlineDownload

Rule:

- If offlineDownload == true, add **8%**.

VideoSubscription

Fields:

- videoQuality
- multiScreen

Rule:

- If multiScreen == true, add **15%**.

Methods

ShowSubscriptionInfo()

CalculateFinalPrice()

IsPremiumSubscription() → price **> 2000**

Main Method:

```
class Program
{
  static void Main(string[] args)
  {
    Subscription[] subs = new Subscription[3];

    subs[0] = new MusicSubscription("S-101","Arif",1000,12,"High",true);
    subs[1] = new VideoSubscription("S-102","Lamia",1500,6,"4K",true);
    subs[2] = new MusicSubscription("S-103","Rafi",900,6,"Standard",false);

    foreach(Subscription s in subs)
    {
      s.ShowSubscriptionInfo();
      Console.WriteLine();
    }
  }
}
```

Output tracing:

141. Output Tracing

```
using System;
class Test
{
  int x;
  public Test()
  {
    x = 5;
    Console.WriteLine(++x);
  }

  public static void Main()
  {
    Test t = new Test();
    int[] a = {2,4,6};
```

```
    foreach(int i in a)
        Console.Write(i + " ");
  }
}
```

142. Output tracing

```
using System;

class Demo
{
  static void Change(ref int a)
  {
    a = a + 10;
  }

  static void Main()
  {
    int x = 5;
    Change(ref x);
    Console.WriteLine(x);
  }
}
```

143. Output tracing

```
using System;

class Test
{
  static void Calc(int a, int b, out int sum)
  {
    sum = a + b;
  }
```

```
    static void Main()
    {
        int s;
        Calc(5,7,out s);
        Console.WriteLine(s);
    }
}
```

144. Output tracing

```
using System;

class A
{
    public A()
    {
        Console.WriteLine("A Constructor");
    }
}

class B : A
{
    public B()
    {
        Console.WriteLine("B Constructor");
    }
}

class Test
{
    static void Main()
    {
        B obj = new B();
```

```
    }
}
```

145. Output tracing

```
using System;

class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal sound");
    }
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks");
    }
}

class Test
{
    static void Main()
    {
        Animal a = new Dog();
        a.Speak();
    }
}
```

146. Output tracing:

```
using System;
```

```
class Student
{
   private int age;

   public int Age
   {
      get { return age; }
      set { age = value; }
   }
}

class Test
{
   static void Main()
   {
      Student s = new Student();
      s.Age = 20;
      Console.WriteLine(s.Age);
   }
}
```

147. Output tracing:

```
using System;

class Test
{
   static void Main()
   {
      int[] a = {1,2,3,4};
      int sum = 0;
```

```
    foreach(int i in a)
      sum += i;

    Console.WriteLine(sum);
  }
}
```

148. Output tracing:

```
using System;

class Test
{
  static int Add(params int[] numbers)
  {
    int sum = 0;
    foreach(int n in numbers)
      sum += n;
    return sum;
  }

  static void Main()
  {
    Console.WriteLine(Add(1,2,3));
  }
}
```

149. Output tracing

```
using System;

class Car
{
  public string brand = "BMW";
}
```

```
class Test
{
   static void Main()
   {
      Car c = new Car();
      Console.WriteLine(c.brand);
   }
}
```

150. Output tracing:

```
using System;

class Test
{
   int x;

   public Test()
   {
      x = 5;
   }

   public Test(int a)
   {
      x = a;
   }

   static void Main()
   {
      Test t1 = new Test();
      Test t2 = new Test(10);
```

```
        Console.WriteLine(t1.x + " " + t2.x);
    }
}
```

151. Output tracing:

```
using System;

class Device
{
    protected int units;
    protected string brand = "Unknown";

    public Device()
    {
        Console.WriteLine("Device Default");
        units = 10;
        Console.WriteLine(++units);
    }

    public Device(int u)
    {
        units = u + 5;
        Console.WriteLine(units);
    }

    public virtual string GetInfo()
    {
        return brand + ":" + units;
    }
}

class Tablet : Device
```

```
{
   int year = 2023;

   public Tablet(int u, string b) : base(u)
   {
      brand = b;
      Console.WriteLine(year - u);
      Show();
   }

   void Show()
   {
      int[] x = {100,200,300,400};
      foreach(int i in x)
      {
         if(i==300) continue;
         Console.Write(i + " ");
      }
      Console.WriteLine();
   }

   public override string GetInfo()
   {
      return brand + "-" + (units + year);
   }
}

class Test
{
   static void Main()
   {
```

```
        Device d = new Device();
        Console.WriteLine(d.GetInfo());

        Tablet t = new Tablet(5,"Lenovo");
        Console.WriteLine(t.GetInfo());
    }
 }
```

152. Output tracing:

```
using System;

class Product
{
   protected int price = 100;
   protected string name = "Item";

   public Product()
   {
      Console.WriteLine("Product Created");
      price += 50;
   }

   public Product(int p)
   {
      price = p;
      Console.WriteLine(price);
   }

   public virtual void Display()
   {
      Console.WriteLine(name + "-" + price);
```

```
    }
}

class Laptop : Product
{
    int warranty = 3;

    public Laptop(int p,string n) : base(p)
    {
        name = n;
        Console.WriteLine(p * warranty);
        Specs();
    }

    void Specs()
    {
        int[] a = {1,2,3,4,5};

        foreach(int i in a)
        {
            if(i==4) break;
            Console.Write(i+" ");
        }

        Console.WriteLine();
    }

    public override void Display()
    {
        Console.WriteLine(name + ":" + (price + warranty));
    }
```

```
}

class Test
{
   static void Main()
   {
      Product p = new Product();
      p.Display();

      Laptop l = new Laptop(500,"HP");
      l.Display();
   }
}
```

153. Output tracing:

```
using System;

class A
{
   protected int x = 5;

   public A()
   {
      Console.WriteLine("A Constructor");
      x++;
   }

   public virtual int Calc()
   {
      return x * 2;
   }
}
```

```
class B : A
{
   int y = 10;

   public B()
   {
      Console.WriteLine("B Constructor");
      y += x;
   }

   public override int Calc()
   {
      return x + y;
   }
}

class Test
{
   static void Main()
   {
      A obj = new B();
      Console.WriteLine(obj.Calc());
   }
}
```

154. Output tracing:

```
using System;

class Numbers
```

```
{
   protected int total = 0;

   public Numbers()
   {
      Console.WriteLine("Numbers Ready");
   }

   public virtual void Add(params int[] n)
   {
      foreach(int i in n)
         total += i;

      Console.WriteLine(total);
   }
}

class AdvancedNumbers : Numbers
{
   public override void Add(params int[] n)
   {
      foreach(int i in n)
         total += i * 2;

      Console.WriteLine(total);
   }
}

class Test
{
   static void Main()
```

```
  {
    Numbers n = new AdvancedNumbers();
    n.Add(1,2,3);
  }
}
```

155. Output tracing:

```
using System;

class Account
{
  protected double balance = 500;

  public Account()
  {
    Console.WriteLine("Account Created");
  }

  public virtual void Deposit(double a)
  {
    balance += a;
    Console.WriteLine(balance);
  }
}

class Saving : Account
{
  double interest = 50;

  public Saving()
  {
    Console.WriteLine("Saving Account");
```

```
    }

    public override void Deposit(double a)
    {
        balance += a + interest;
        Console.WriteLine(balance);
    }
}

class Test
{
    static void Main()
    {
        Account a = new Saving();
        a.Deposit(200);
    }
}
```

156. Output tracing:

```
using System;

class Student
{
    protected int id;
    protected string name="Unknown";

    public Student()
    {
        Console.WriteLine("Student Created");
    }

    public Student(int i)
```

```
    {
        id=i;
        Console.WriteLine(id);
    }

    public virtual string Info()
    {
        return name+"-"+id;
    }
}

class Graduate : Student
{
    int year=2024;

    public Graduate(int i,string n):base(i)
    {
        name=n;
        Console.WriteLine(year-i);
        Show();
    }

    void Show()
    {
        int[] a={11,22,33,44};

        foreach(int x in a)
        {
            if(x==33) continue;
            Console.Write(x+" ");
        }
```

```
        Console.WriteLine();
    }

    public override string Info()
    {
        return name+":"+(id+year);
    }
}

class Test
{
    static void Main()
    {
        Graduate g=new Graduate(10,"Alice");
        Console.WriteLine(g.Info());
    }
}
```

157. Output tracing:

```
using System;

class Base
{
    protected int a=5;

    public Base()
    {
        Console.WriteLine("Base");
        a+=5;
    }
```

```
    public virtual int Show()
    {
        return a;
    }
}

class Derived:Base
{
    int b=10;

    public Derived()
    {
        Console.WriteLine("Derived");
        b+=a;
    }

    public override int Show()
    {
        return a+b;
    }
}

class Test
{
    static void Main()
    {
        Base obj=new Derived();
        Console.WriteLine(obj.Show());
    }
}
```

158. Output tracing:

```
using System;

class ArrayTest
{
   protected int[] data={2,4,6,8};

   public virtual void Show()
   {
      foreach(int x in data)
         Console.Write(x+" ");

      Console.WriteLine();
   }
}

class Filter:ArrayTest
{
   public override void Show()
   {
      foreach(int x in data)
      {
         if(x==6) continue;
         Console.Write(x+" ");
      }

      Console.WriteLine();
   }
}

class Test
{
```

```
    static void Main()
    {
        ArrayTest obj=new Filter();
        obj.Show();
    }
}
```

159. Output tracing:

```
using System;

class Calc
{
    protected int x=5;

    public virtual int Add(int a)
    {
        return a+x;
    }
}

class AdvCalc:Calc
{
    public override int Add(int a)
    {
        return a*x;
    }
}

class Test
{
    static void Main()
```

```
  {
    Calc c=new AdvCalc();
    Console.WriteLine(c.Add(4));
  }
}
```

160. Output tracing:

```
using System;

class Machine
{
  protected int power=100;

  public Machine()
  {
    Console.WriteLine("Machine Start");
  }

  public virtual void Run()
  {
    Console.WriteLine(power);
  }
}

class Robot:Machine
{
  int speed=5;

  public Robot()
  {
    Console.WriteLine("Robot Ready");
```

```
    }

    public override void Run()
    {
        Console.WriteLine(power*speed);
    }
}

class Test
{
    static void Main()
    {
        Machine m=new Robot();
        m.Run();
    }
}
```

161. Practice Problem

```
using System;

class Device
{
    protected int units;
    protected string brand = "Generic";

    public Device()
    {
        Console.WriteLine("Device Start");
        units = 3;
        Console.WriteLine(++units);
    }
```

```
    public Device(int u)
    {
        units = u + 2;
        Console.WriteLine(units);
    }

    public virtual string Info()
    {
        return brand + ":" + units;
    }
}

class Laptop : Device
{
    int warranty = 2;

    public Laptop(int y, string b) : base(y)
    {
        brand = b;
        Console.WriteLine(warranty * y);
        Specs();
    }

    void Specs()
    {
        int[] a = {101,102,103,104};

        foreach(int x in a)
        {
            if(x==103) continue;
```

```
        Console.Write(x+" ");
      }

      Console.WriteLine();
    }

    public override string Info()
    {
      return brand + "-" + (units + warranty);
    }
}

class Test
{
    static void Main()
    {
      Device d = new Device();
      Console.WriteLine(d.Info());

      Laptop l = new Laptop(4,"Dell");
      Console.WriteLine(l.Info());
    }
}
```

162. Practice Problem

```
using System;

class A
{
    protected int x = 5;

    public A()
```

```
    {
        Console.WriteLine("A Constructor");
        x++;
    }

    public virtual int Show()
    {
        return x * 2;
    }
}

class B : A
{
    int y = 10;

    public B()
    {
        Console.WriteLine("B Constructor");
        y += x;
        Test();
    }

    void Test()
    {
        int[] arr = {1,2,3,4,5};

        foreach(int i in arr)
        {
            if(i==4) break;
            Console.Write(i+" ");
        }
```

```
        Console.WriteLine();
    }

    public override int Show()
    {
        return x + y;
    }
}

class Test
{
    static void Main()
    {
        A obj = new B();
        Console.WriteLine(obj.Show());
    }
}
```

163. Practice Problem

```
using System;

class Machine
{
    protected int power;

    public Machine()
    {
        power = 50;
        Console.WriteLine(power);
    }
```

```
    public Machine(int p)
    {
        power = p + 10;
        Console.WriteLine(power);
    }

    public virtual int Run()
    {
        return power * 2;
    }
}

class Robot : Machine
{
    int speed = 5;

    public Robot(int p) : base(p)
    {
        Console.WriteLine(speed + p);
        Show();
    }

    void Show()
    {
        int[] data = {2,4,6,8};

        foreach(int d in data)
        {
            if(d==6) continue;
            Console.Write(d+" ");
        }
```

```
    Console.WriteLine();
  }

  public override int Run()
  {
    return power + speed;
  }
}

class Test
{
  static void Main()
  {
    Robot r = new Robot(20);
    Console.WriteLine(r.Run());
  }
}
```

164. Practice Problem

```
using System;

class Student
{
  protected int id;
  protected string name="None";

  public Student()
  {
    id = 10;
    Console.WriteLine(id);
```

```
    }

    public virtual string Info()
    {
        return name + "-" + id;
    }
}

class Graduate : Student
{
    int year = 2025;

    public Graduate(string n)
    {
        name = n;
        Console.WriteLine(year - id);
        Display();
    }

    void Display()
    {
        int[] x = {5,10,15,20};

        foreach(int i in x)
        {
            if(i==15) continue;
            Console.Write(i+" ");
        }

        Console.WriteLine();
    }
```

```
  public override string Info()
  {
    return name + ":" + (year + id);
  }
}

class Test
{
  static void Main()
  {
    Graduate g = new Graduate("Alice");
    Console.WriteLine(g.Info());
  }
}
```

165. Practice Problem

```
using System;

class Shape
{
  protected int side = 4;

  public Shape()
  {
    Console.WriteLine("Shape Created");
  }

  public virtual int Area()
  {
    return side * side;
  }
```

```
}

class Square : Shape
{
   int factor = 2;

   public Square()
   {
      Console.WriteLine("Square Ready");
      Check();
   }

   void Check()
   {
      int[] a = {3,6,9,12};

      foreach(int i in a)
      {
         if(i==9) break;
         Console.Write(i+" ");
      }

      Console.WriteLine();
   }

   public override int Area()
   {
      return side * side * factor;
   }
}
```

```
class Test
{
   static void Main()
   {
      Shape s = new Square();
      Console.WriteLine(s.Area());
   }
}
```